Vorssaint 能力接入 PinWall

横纵分析法深度研究报告

作者: PinWall 技术可行性评估

> 研究日期：2026-10-26 | 范围：PinWall Desktop | 结论：可行，按能力独立实现，先做官方可选插件

结论与范围

PinWall 可以吸收 Vorssaint 的部分能力，适合采用“插件界面 + 宿主原生能力”的结构。已有 Widget 安装、iframe 展示和消息桥可以复用，但它们仍是实验基础，不能直接承载全部系统工具，也尚不足以隔离任意第三方代码。

推荐独立实现功能，首期验证防休眠、基础系统监控和一次性“剪贴板转便签”。插件均默认关闭，属于项目约定的 P2 可选能力。没有启用插件时，核心便签的启动、离线操作、提醒和数据恢复保持现有路径。

本方案基于上游公开文档、部分 Swift 源码及本地 desktop 代码静态检查。没有编译或运行 Vorssaint，没有完成原生交互、硬件兼容和性能实测。难度分级是工程判断，不能当作交付承诺。

参考项目到底是什么

Vorssaint 是原生 macOS 工具合集。README 列出了按应用音量、音频输出切换、系统监控、风扇控制、窗口布局、Dock 预览、鼠标键盘增强、剪贴板历史、文件暂存、截图录屏、离线 OCR、媒体处理、应用卸载与防休眠等能力。[S1]

其 Package.swift 指定 Swift 5.9、macOS 14，目标是一个原生可执行程序，含 HIDEventSystem 与 VMStatisticsCompat 系统库；贡献指南说明构建脚本直接构建和签名应用，不依赖第三方 Swift 包。README 的当前支持范围为 Apple Silicon、macOS 14 及以上。[S2][S3]

因此，上游不是可以直接导入的 JavaScript 工具包。SwiftUI 界面需要重新制作；跨应用控制、屏幕捕获、音频、输入事件等能力需要平台适配。把整个应用当作 HTML Widget 包装，无法获得这些能力。

演进中值得借鉴的设计

当前 FeatureCatalog 把能力划分为窗口/Dock、鼠标键盘、剪贴板文件、声音、能耗显示、工具、灵动岛和监控等组。功能有稳定 ID，“是否安装可用”独立于功能内部开关；目录注释明确要求不可用功能停止服务，并在下一次启动时不实例化，重新启用则保留原设置。[S4]

ClipboardHistoryService 进一步显示：它有明确 start/stop、剪贴板检查与保留期定时器，并用独立访问通道避免剪贴板服务器阻塞主线程；禁用时释放定时器。代码也排除带保密标记的剪贴板内容。这里值得借鉴的是生命周期和阻塞隔离，而不只是功能列表。[S5]

CHANGELOG 的迭代涉及剪贴板阻塞、窗口枚举线程、显示器重连、音频清理等修复，说明系统工具的主要工作量会持续发生在权限、后台生命周期和系统边界处。[S6] 本次未完整重建项目提交历史，不推断创始背景或早期发布时间。

能力分级与建议

下表“低/中/高”是相对于当前 PinWall 的开发与维护复杂度，不表示已有实现。

能力	插件形态与宿主工作	复杂度	建议
链接清理、文本片段库、按需计算	本地前端逻辑；如需复制结果，再接受控剪贴板写入	低	可作为无原生扩展的先导；不含全局文本自动展开
防休眠	小卡片控制；原生电源断言和计时释放	中	首期；只做普通防休眠，不做合盖和管理员规则
基础系统监控	卡片展示；原生统一采样 CPU、内存和电池	中	首期；先保证指标准确，再扩展历史图
剪贴板转便签	用户点击后读取文本，再创建便签	中	首期；暂不持续记录全部剪贴板
剪贴板历史、文件暂存 Shelf	原生后台监听、独立数据存储、文件访问和拖放	中高	第二期；设置保留期、暂停和清空入口
截图、屏幕 OCR、取色	屏幕选择界面；屏幕权限与本地识别/捕获	中高	第二期；OCR 结果可转便签
本地图片处理	用户选文件；浏览器或原生图像处理、受控导出	中	第二期；视频处理另做资源预算
音频混音与每应用路由	系统音频服务、设备变化、实时处理	高	单独立项；不适合作为验证插件机制的首个功能
窗口布局、Dock 预览与切换器	Accessibility、窗口枚举、预览与快捷键	高	后期；先验收与 PinWall 点击穿透、窗口层级的共存
鼠标键盘增强与自动文本展开	全局事件监听/注入、应用排除和失败恢复	高	后期独立评估；可能影响全系统输入

能力	插件形态与宿主工作	复杂度	建议
风扇、XDR 亮度、合盖防休眠	硬件、系统版本或特权适配	高	暂缓；基础架构阶段不引入
清理器、卸载器、Homebrew、结束进程	广域文件/应用/进程操作及失败回滚	高	暂缓，与便签工作台主线关联较弱

灵动岛可以借鉴其信息呈现方式，但 PinWall 已有桌面墙，不必再建立一套摄像头刘海附近的窗口体系。Scratchpad 与现有便签高度重合，应复用便签而不是再建第二套笔记存储。

PinWall 当前具备什么

环节	已检查到的实现	限制
Manifest	ID、入口、图标、分类、权限、尺寸、设置	没有区分按需工具与后台服务的契约
安装	本地目录安装、官方列表安装，复制到 AppData	更新安装失败回滚和资源配额需补
路径检查	ID 校验、拒绝路径逃逸、复制拒绝符号链接	不能替代运行时隔离
渲染	WidgetFrame + iframe；主墙为右侧面板	按需大工具界面与生产多文件资源加载需验证
通信	postMessage → widgetBridge → store / invoke / fetch	请求参数缺乏严格运行时协议校验
存储	插件实例设置持久化、独立插件 JSON 数据文件	读改写并发、损坏保护、迁移与配额需补

具体入口：src/types/index.ts:258、src/services/widgetLoader.ts:31、src/components/WidgetFrame.tsx:17、src/services/widgetBridge.ts:144、src/stores/widgetStore.ts:163、src-tauri/src/widgets.rs:249。[L1-L6]

不能把目录上的项目都视为完成的系统能力。当前通知 Bridge 只返回成功，并未实际送达；settings 返回空对象；主题是固定色值。原生系统信息只有部分电池/内存查询，媒体状态为示例数据，控制接口为占位。[L3][L6]

当前 network Bridge 使用宿主 WebView 的 fetch，且只接受 JSON，不是通用下载器，也不等于有任意 HTTP/文件能力。现有插件没有正式的剪贴板、屏幕捕获、任意文件、全局输入监听或 shell API。

推荐架构

调用链：插件 UI → 版本化 Bridge → 宿主授权与参数校验 → 原生能力服务 → macOS API。事件从原生服务经宿主路由回已授权的实例。

插件 UI 使用现有 widget.json 与 Web 界面。适合卡片的能力留在侧栏；OCR、历史记录等内容较多的工具，在按需打开的大面板呈现。先复用现有窗口内面板；确需独立窗口再增加受限 capability，并执行窗口 QA。

Bridge 为每一种能力定义明确的请求、响应、错误和取消协议。建议的能力名如 power.keepAwake、system.metrics.read、clipboard.readTextOnce、cards.create，是未来提案，当前尚不存在。不要开放一个任意 command、invoke、shell 或脚本执行入口。

授权 同时检查插件声明、用户授予记录、官方来源身份、请求参数及系统权限状态。插件 ID 由宿主根据实际 iframe/会话绑定，不信任请求自行提交的 ID。“official” 字段不能用作信任凭据。安装本地包也需预览权限并授权。

原生能力服务 统一管理资源、后台任务和订阅。基础指标在一个采样服务中取得后分发，多个卡片共享结果；可见订阅全部释放后停止不必要的采样。防休眠有计时到期、停用、卸载和退出清理；有意在界面隐藏后继续的服务，以用户启用状态维持，而不依赖 iframe 活着。

平台实现 优先 Rust 调用稳定平台 API。需要 Apple 原生框架时，可通过 Objective-C/Swift 的小型桥接实现；只有依赖和故障隔离确有需要时才引入受控 helper。首期不动态加载第三方 dylib，不允许插件携带任意原生可执行代码。

macOS 的系统授权通常授予 PinWall 应用，插件级限制仍需 PinWall 自己执行。Tauri capability 也不天然提供每个插件的用户授权。签名身份、权限用途描述和升级后的权限保留需要作为原生能力发布条件。

Web 插件可以独立更新界面和算法；新增原生能力仍需要发布 PinWall 更新。这是首期有意保留的边界。

开放第三方前的基础门槛

当前已安装插件读取 HTML 后通过 srcDoc 展示，iframe 同时设置 allow-scripts 与 allow-same-origin；srcDoc 与宿主同源时，这个组合允许访问父页，不能依赖 Bridge 权限作为可靠隔离边界。配置中的 CSP 为 null，资源范围为整个 AppData。[L2][L7]

应使用独立源或 opaque sandbox，避免插件取得父页 DOM 和宿主 IPC；建立按插件限制的资源及网络策略。单纯移除一个 sandbox token 不是完整方案，还需验证生产环境静态资源、初始化通信和消息来源校验。

读取入口 HTML 不代表 React/Vue 多文件构建目录可以直接使用。首期可采用经过审查的单文件 HTML；以后通过受控资源协议支持 JS/CSS/图片，限制访问范围并避免插件之间共享数据。

桥接增加版本、运行时校验、未知方法拒绝、请求大小限制、限流、超时及取消。插件数据增加原子更新、损坏文件保留、schema version 和迁移策略。安装采用暂存、校验、原子切换，失败恢复旧版本；停用、卸载和授权撤销立即释放订阅和服务。

以上是开放插件所需的基础。首期仅随应用发布少量审查过的官方插件，但在扩展敏感原生能力时仍应同步加固隔离，不能只依赖来源可信。

首期实施顺序

1. **补插件基础**：严格 Bridge 协议、会话绑定、授权模型、沙箱与资源策略。明确 UI 挂载、功能启用、后台服务三者的生命周期。新增字段要给旧插件默认行为并测试迁移。
2. **防休眠插件**：仅普通防休眠，提供时长、开始、停止和剩余时间。验证隐藏界面后计时、停用后释放、重复点击幂等、退出与睡眠唤醒。
3. **基础监控插件**：CPU、内存、电池，统一采样。优先修正现有数据解析，不把温度、风扇、每进程能耗承诺进首版；无法读取时显示“不支持/读取失败”。
4. **剪贴板转便签**：仅在用户点击时读文本，预览后创建便签。不要给插件全部便签读权限；把现有粗粒度 cards 能力细化，并修正创建返回值契约。持续历史、自动粘贴和文件/图片记录留后续。
5. **验收后再扩展**：依据用户使用情况添加剪贴板历史、OCR 转便签、图片处理。每新增一个后台能力，都先定义停止条件、数据范围、资源预算和拒绝权限后的界面状态。

低成本前导可以先做 URL 清理，验证“安装 → 打开 → 使用 → 停用”，但它无法验证后台原生服务。不要用纯前端演示通过替代原生插件方案的验证。

验收与影响范围

代码范围优先限定 desktop：类型契约、WidgetFrame、widgetBridge、widgetStore、原生 widgets/新增能力服务、权限说明与设置界面、相关测试。不要顺手重构 frontend、website、marketplace 或 backend。现有 widget-sdk 与 widgets 为相邻独立目录，若需要发布 SDK/官方插件包，再单独明确扩展范围。

先补协议和服务测试，再接 UI。重点覆盖未授权请求、伪造插件身份、越界访问、服务重复启动/停止、后台任务取消、存储损坏和生产资源加载。运行 pnpm test:run、pnpm build 与必要的 cargo test；如新增窗口、快捷键和通知，按 RELEASE_QA 执行真实 macOS 检查。浏览器 E2E 不能替代 TCC、屏幕权限与窗口层级测试。

性能要先测无插件基线，再以同机同场景比较：核心启动时间、空闲 CPU、唤醒频率、内存、10+ 便签拖拽、关闭/卸载后的活动任务数。具体阈值在基线实测后设定；本次没有运行性能测试，不虚报数值。

本轮仅新增调研产物，没有修改产品行为和持久化 schema，因此没有运行构建或测试，也不声称插件已经落地。

开源许可路线

上游声明 GPL-3.0-or-later，且名称、图标、外观另受商标说明约束；PinWall 根 LICENSE 当前是 MIT。[S1][S3][L8]

推荐把上游作为功能、行为和边界参考，独立制作 PinWall 界面并基于系统 API 实现。不要直接复制 Swift 代码、资源和品牌视觉。

如果决定复用上游 GPL 代码，MIT 代码可进入符合 GPL 要求的组合，但不能把复制来的代码继续仅标为 MIT；分发派生或组合程序时需要评估相应 GPL 义务、源码提供和许可声明。是否构成一个组合程序取决于实际集成方式，不能仅靠“插件”或“独立进程”的命名得出结论。该路线应在选型时明确，而不是等打包前再处理。

判断与后续选择

最可能成功的路线，是把 PinWall 扩展成“便签 + 少量可选桌面工具”：防休眠维持工作会话，监控提供状态，剪贴板/OCR 将临时信息沉淀为便签。每项都强化已有桌面工作台。

最大的实施风险是一次性追全功能。音频、输入、窗口、清理和硬件控制会让项目承担多个系统工具的维护成本，并扩大对核心便签稳定性的影响。最乐观的路线是先把少量官方能力做稳，再用成熟的授权、隔离和生命周期逐步接纳第三方 Web 插件。

建议下一步的开发范围为：先补插件基础，并落地防休眠插件作为完整样板。它足够小，又能验证 UI、Bridge、授权、原生服务、计时与退出清理的完整链路。

信息来源

所有远程资料访问于 2026-10-26。main 分支会变化；本次部分源码固定在提交 746239b0220571d6694ffacae79f3eb9dcd09193。文件树下载发生超时，仅用于定位实际读到的文件，未视作完整目录审计。

- [S1] README：https://github.com/vorssaint/vorssaint-utils/blob/main/README.md
- [S2] Swift 包配置：https://github.com/vorssaint/vorssaint-utils/blob/main/Package.swift
- [S3] 贡献与构建约定：https://github.com/vorssaint/vorssaint-utils/blob/main/CONTRIBUTING.md
- [S4] 功能目录：https://github.com/vorssaint/vorssaint-utils/blob/746239b0220571d6694ffacae79f3eb9dcd09193/Sources/Vorssaint/Core/FeatureCatalog.swift
- [S5] 剪贴板服务：https://github.com/vorssaint/vorssaint-utils/blob/746239b0220571d6694ffacae79f3eb9dcd09193/Sources/Vorssaint/Services/Clipboard/ClipboardHistoryService.swift
- [S6] 更新记录：https://github.com/vorssaint/vorssaint-utils/blob/main/CHANGELOG.md
- [S7] 权限说明：https://github.com/vorssaint/vorssaint-utils/blob/main/docs/PERMISSIONS.md
- [L1] desktop/src/types/index.ts:258；src/services/widgetLoader.ts:31
- [L2] desktop/src/components/WidgetFrame.tsx:46、251
- [L3] desktop/src/services/widgetBridge.ts:144、222、234、338
- [L4] desktop/src/stores/widgetStore.ts:163
- [L5] desktop/src/data/officialWidgets.ts:8
- [L6] desktop/src-tauri/src/widgets.rs:249、346、379
- [L7] desktop/src-tauri/tauri.conf.json:51；src-tauri/capabilities/default.json
- [L8] PinWall/LICENSE（根目录）；desktop/AGENTS.md；desktop/README.md

方法说明：参考横纵分析法，以当前源码中的架构演进线索解释设计取舍，并横向对照 PinWall 的能力、约束和接入方式。本报告按技术可行性任务收敛，不替代完整历史研究或原生原型验证。